

Document Intelligence Platform

AI-Powered Document Extraction, Validation & API Platform

The Problem

Financial teams receive invoices and statements as native or scanned PDF / JPG / PNG files, in inconsistent layouts — manual extraction is slow and error-prone.



Inconsistent layouts

Every vendor / bank formats statements differently



OCR errors

Scanned documents introduce noise and misreads



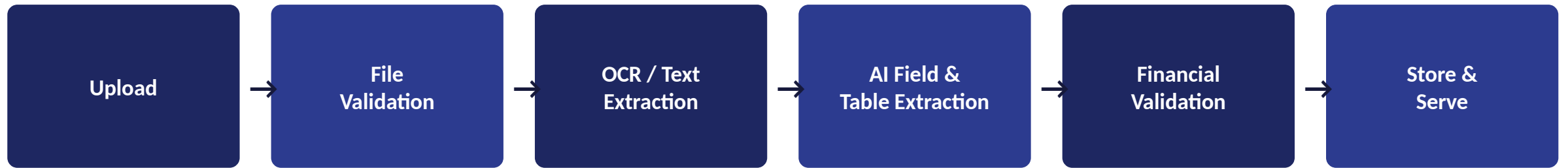
Manual reconciliation

Slow, inconsistent, and error-prone at scale

Goal

Build a reusable Document Intelligence service that extracts complete information, validates key financial relationships, and exposes results through a deployed dashboard and REST APIs.

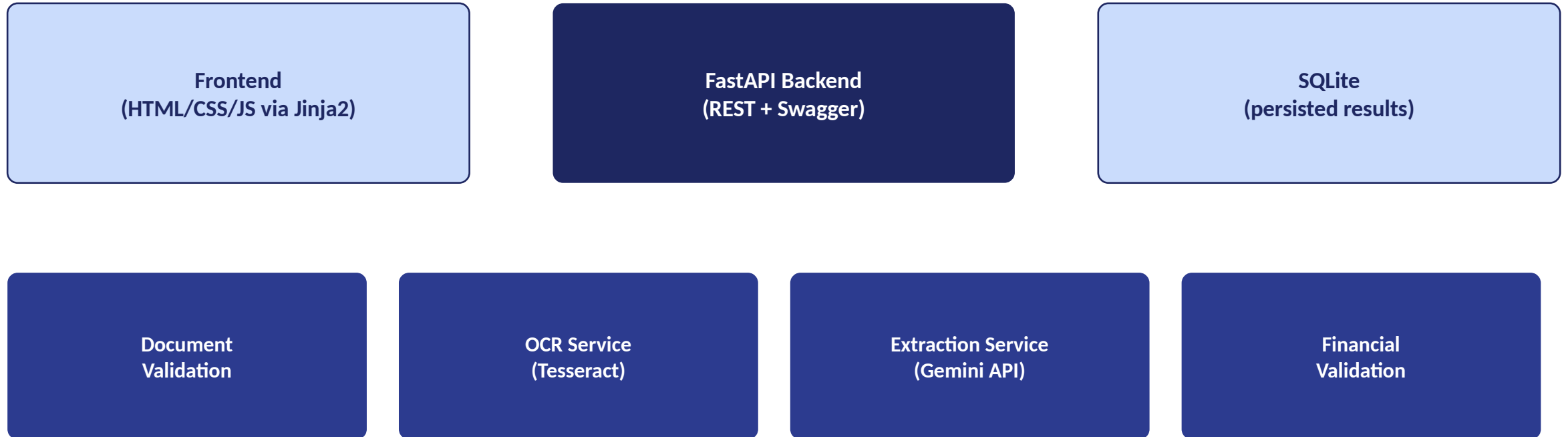
Processing Pipeline



Every stage fails gracefully: unsupported/corrupt files, OCR failures, and LLM failures are all caught and returned as controlled, structured errors — never a raw stack trace.

PASS / FAILED status, evidence (source text + page number), and processing metadata accompany every result.

Architecture Overview



services/ layer — each concern isolated behind its own module, orchestrated by document_service.py

Deployed on Render — separate live URLs for frontend, API, and Swagger docs (/docs).

Technology Stack & Rationale

Backend	FastAPI	Async, automatic Swagger/OpenAPI docs, native Pydantic validation
Database	SQLite (SQLAlchemy ORM)	Zero-config persistence; swappable to Postgres via one env var
OCR	Tesseract + pdf2image	Free, local, no API rate limits under a 3-day deadline
AI Extraction	Google Gemini API	Free-tier API key, no billing required, for a fast prototype
Frontend	Jinja2 + vanilla JS	Meets the no-separate-frontend-stack requirement, fast to build
Deployment	Render (free tier)	Single web service serves both API and templated frontend

REST API Design

POST	<code>/api/v1/documents/process</code>	Upload & process a PDF/JPG/PNG document
GET	<code>/api/v1/documents/{document_name}</code>	Retrieve the latest structured result
GET	<code>/api/v1/documents</code>	List processed documents for the dashboard
GET	<code>/api/v1/health</code>	Health check for the deployed service

Every response is consistent, structured JSON:

```
document_name · document_type · processing_status  
file_validation · extracted_data (value + evidence) · validation (checks, PASS/FAIL/NOT_APPLICABLE)  
processing_metadata (OCR used, model, timing)
```

Financial Validation Logic

Every check reports: formula, input operands, calculated value, reported value, variance, and status.

Invoice	<code>subtotal + tax - discount ≈ total_amount</code>	PASS
Balance Sheet	<code>total_liabilities + total_equity ≈ total_assets</code>	PASS
Profit & Loss	<code>revenue - cost_of_sales ≈ gross_profit → ... → net_profit</code>	FAIL
Cash Flow	<code>operating + investing + financing + FX ≈ net_change_in_cash</code>	PASS

Missing inputs → NOT_APPLICABLE (never guessed). Parenthesized values are treated as negative. Tolerance is configurable.

Frontend & Dashboard

Upload & Process

Select document type, upload PDF/JPG/PNG, trigger processing

Dashboard

All processed documents: name, type, status, processed time

Result View

Extracted key-value fields, line-item tables, validation results

Raw JSON

Full structured response viewable for evaluator inspection

Database-backed dashboard connected to the deployed API — results persist across sessions for evaluation.

Known Limitations & Production Roadmap

Current Limitations

- Synchronous processing — large batches would need a queue
- Confidence scoring omitted (optional per spec)
- Single-model OCR fallback (Tesseract only)
- SQLite is fine for evaluation, not for concurrent production load

For Production

- Async processing queue (Celery/RQ) with webhook/polling status
- Postgres + object storage (S3) for documents and results
- Multi-provider OCR fallback (Cloud Vision / Document AI)
- Human-in-the-loop review queue for low-confidence extractions

Goal met: a reliable, structured, testable and deployable AI engineering prototype — not an enterprise platform.